BGIN0011

BGIN IKP Working Group

Study Report: Zero-Knowledge Proofs (ZKPs) – Technology and Applications

- BGIN is a multi-stakeholder discussion body / distributed think tank.

bg in

blockchain
governance
initiative
network

The Global Network for Blockchain Stakeholders™

© 2023 BGIN (Blockchain Governance Initiative Network) is registered as BN Association, a Japanese general association with its principal place of business at Shiba Building, 704, 4-7-6, Shiba, Minato-ku, Tokyo. All rights reserved.

The know-how described in this document was made available from contributions from various sources, including members of the BGIN and others. Although the BGIN has taken steps to help ensure that the know how is available for distribution, it takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the know how described in this document or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any independent effort to identify any such rights. BGIN and the contributors to this document make no (and hereby expressly disclaim any) warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to this document, and the entire risk as to implementing this document is assumed by the implementer. The BGIN Intellectual Property Rights policy requires contributors to offer a patent promise not to assert certain patent claims against other contributors and against implementers. BGIN invites any interested party to bring to its attention any copyrights, patents, patent applications, or other proprietary rights that may cover technology that may be required to practice this document.

BGIN's Multi-stakeholder Approach

BGIN's distinctive multi-stakeholder approach provides a uniquely valuable foundation for advancing blockchain governance research. As a neutral facilitator with global reach and technical credibility, BGIN orchestrates collaborative processes that balance diverse interests across jurisdictional boundaries while maintaining focus on public good outcomes. This approach enables candid discussion of sensitive methodological issues that might be challenging in more commercially oriented environments. The organization's commitment to transparent development with broad stakeholder input, combined with its educational focus that translates complex technical concepts for varied audiences, creates an inclusive ecosystem where best practices can emerge organically. Through regular workshops, discussion forums, and public consultations, BGIN harnesses collective expertise spanning technical, legal, and regulatory domains to develop frameworks that are both technically sound and practically applicable across diverse global contexts. This collaborative methodology ensures that emerging standards benefit from comprehensive perspectives while remaining adaptable to evolving technological and regulatory landscapes.

BGIN SR 0011

Study Report: Zero-Knowledge Proofs (ZKPs) – Technology and Applications

Author: Leona Hioki (BGIN) and contributors
Editor: Mitchell Travers (BGIN)
Working Group: IAM, Key Management and Privacy Working Group

This study is a work product of the IAM, Key Management and Privacy Working Group of BGIN.

©2025 BGIN (Blockchain Governance Initiative Network) is registered as BN Association, a Japanese general association with its principal place of business at Shiba Building, 704, 4-7-6, Shiba, Minato-ku, Tokyo. All rights reserved.

Executive Summary

This report consolidates modern zero-knowledge proof (ZKP) practice for blockchain applications, examining arithmetization choices (R1CS, Plonkish, AIR, CCS), proving backends (pairing/KZG, IPA, FRI/STARK, sumcheck-centric), recursion and folding/IVC (Nova/SuperNova/HyperNova), zkVMs, and post-EIP-4844 rollup economics. It provides concrete derivations (Groth16/QAP, KZG, sumcheck, FRI), proof size and cost estimates, and security considerations for production deployments.

Zero-knowledge proofs enable one party (the prover) to convince another party (the verifier) that a statement is true without revealing any information beyond the validity of the statement itself. Since their theoretical foundation in 1985 and practical implementation beginning in 2013, ZKPs have evolved from academic curiosity to production-ready technology powering privacy-preserving transactions, scalable blockchain infrastructure, and verifiable computation systems.

Key Capabilities:

- **Privacy Enhancement**: Privacy-preserving transactions and identity verification without exposing sensitive data
- **Scalability Solutions**: zkRollups enable thousands of off-chain transactions while maintaining on-chain security
- **Verifiable Computation**: Verification of complex computations with minimal overhead

Table of Contents

BGIN SR 0011 ..... エラー! ブックマークが定義されていません。
Study Report: Zero-Knowledge Proofs (ZKPs) – Technology and Applications 3
Executive Summary 3
Table of Contents 4
1. Scope and Audience 6
2. Conventions and Notation 6
3. Terms and Definitions 6
3.1 Zero-Knowledge Proof (ZKP) & NIZK 6
3.2 Finite Fields, Groups, Curves 7
3.3 Commitments, Hashes, Vector Commitments 7
3.4 Arithmetic Circuit 7
3.5 Polynomial Problem 8
3.6 Witness 8
3.7 Instance 8
3.8 Fiat-Shamir Transformation 8
4. Abbreviations and Symbols 8
5. Definition and History of ZKP 9
5.1 Origins 9
5.2 Evolution to Practical Systems 9
6. ZKP Technology Architecture 10
6.1 Two-Layer Architecture 10
6.2 Key Properties 10
7. Frontend Systems: Arithmetization 10
7.1 R1CS (Rank-1 Constraint System) 10
7.2 Plonkish Gates 11
7.3 AIR (Algebraic Intermediate Representation) 11
7.4 CCS (Customizable Constraint Systems) 12
8. Backend Systems: Proving Mechanisms 12
8.1 Pairing-Based Systems (Groth16, PLONK) 12
8.2 IPA (Inner Product Argument / Bulletproofs) 12
8.3 FRI / STARK 13
8.4 Folding / IVC (Nova, HyperNova) 13
9. Polynomial Commitment Schemes 14
9.1 KZG (Kate-Zaverucha-Goldberg) 14
9.2 IPA (Inner Product Argument) 14
9.3 FRI 14
10. Recursion and Folding 14
10.1 Recursive Proof Composition 15
10.2 Folding Schemes (Nova Family) 15
11. Applications 15

11.1 Privacy-Preserving Transactions 15
11.2 zkRollups 16
11.3 Related zkp Solutions (Not zkRollups) 17
11.4 Identity and Credentials 17
11.5 Verifiable Machine Learning (zkML) 17
11.6 Cross-Chain Bridges 18
12. Programming Languages and Libraries 18
12.1 Domain-Specific Languages 18
12.2 Libraries and Frameworks 19
12.3 zkVMs 20
13. Performance Trade-offs 20
13.1 Fundamental Trade-offs 20
13.2 Recursion Capabilities 21
13.3 Practical Considerations 21
14. Security Considerations 21
14.1 Trusted Setup 21
14.2 Parameter Selection 22
14.3 Circuit Design Vulnerabilities 22
14.4 Implementation Vulnerabilities 22
14.5 Audit Requirements 23
15. Mathematical Derivations 23
15.1 Groth16 via QAP (Quadratic Arithmetic Program) 23
15.2 KZG Evaluation Proof 24
15.3 Sumcheck Protocol (Sketch) 24
15.4 FRI Low-Degree Test 24
16. Measurements and Representative Sizes 24
16.1 Proof Sizes (Typical Configurations) 24
16.2 EIP-4844 Blob Economics 25
17. Choosing a Stack 26
18. Ecosystem Snapshot (2024-2025) 26
19. References 27
Foundational Papers 27
Modern Systems 27
Polynomial Commitments 27
zkRollups and Scaling 27
Security 28
Online Resources 28
20. Appendices 28
Appendix A: Glossary 28
Appendix B: System Comparison Tables 29
Appendix C: On-Chain Verification Notes 30
Appendix D: zkRollup Considerations 30
Appendix Z 32
1. Scope and Audience .....エラー! ブックマーク名が指定されていません。

2. Conventions and Notation 33
3. Terms and Definitions (Corrected & Expanded)エラー! ブックマーク名が指定されていません。
3.1 Zero-Knowledge Proof (ZKP) & NIZK .....エラー! ブックマーク名が指定されていません。
3.2 Finite Fields, Groups, Curves .....エラー! ブックマーク名が指定されていません。
3.3 Commitments, Hashes, Vector Commitments 34
13. Choosing a Stack.....エラー! ブックマーク名が指定されていません。
14. Ecosystem Snapshot (2024–2025).....エラー! ブックマーク名が指定されていません。
15. Mathematical Derivations (Concise)...エラー! ブックマーク名が指定されていません。
15.1 Groth16 via QAP.....エラー! ブックマーク名が指定されていません。
15.2 KZG Evaluation Proof.....エラー! ブックマーク名が指定されていません。
15.3 Sumcheck (Sketch).....エラー! ブックマーク名が指定されていません。
15.4 FRI (Low-Degree Test) 52
16. Measurements & Representative Sizes ....エラー! ブックマーク名が指定されていません。
17. On-Chain Verification Notes.....エラー! ブックマーク名が指定されていません。
18. References (Selected) 53
Appendix A — Glossary.....エラー! ブックマーク名が指定されていません。
Appendix B — Comparison Tables .....エラー! ブックマーク名が指定されていません。
Appendix C — Worked Examples (Sketches) 54

---

1. Scope and Audience

Scope: Practical non-interactive proofs in production—zkSNARKs, zkSTARKs, and folding-based IVC—and their use in privacy, scalability, and proofs of computation.

Audience: Protocol engineers, auditors, product teams, blockchain developers, security professionals, and policy makers working with or evaluating ZKP systems.

---

2. Conventions and Notation

We write fields as $\mathbb{F}_q$ (q=p^k), elliptic-curve groups as G1/G2/G_T, and use CRS/SRS for reference strings. Standard cryptographic notation applies throughout.

---

3. Terms and Definitions

3.1 Zero-Knowledge Proof (ZKP) & NIZK

ZKP: A cryptographic protocol satisfying three properties:

- **Completeness**: If the statement is true, an honest prover convinces an honest verifier
- **Soundness** (or knowledge soundness for SNARKs): If the statement is false, no prover can convince the verifier except with negligible probability
- **Zero-knowledge**: The verifier learns nothing beyond the truth of the statement

NIZK: Non-interactive zero-knowledge proof, obtained natively or via Fiat-Shamir transformation (ROM).

3.2 Finite Fields, Groups, Curves

Finite field $\mathbb{F}_q$ has $q=p^k$ elements where:

- characteristic($\mathbb{F}_q$)=p
- Every non-zero element has a multiplicative inverse

Elliptic curves over $\mathbb{F}_q$ form abelian groups:

- Identity element $\mathcal{O}$
- For finite group order n: $n \cdot P = \mathcal{O}$ for any point P
- **Important**: Group order ≠ field characteristic

Elliptic Curve: An equation that can be represented as $y^2 = x^3 + ax + b$ over a finite field. Points P and Q on the elliptic curve have a defined addition operation, making P + Q = R. While direct multiplication P × Q isn't defined, scalar multiplication like P+P=2P and nP (n times P) is defined. Due to the discrete logarithm problem, it's computationally infeasible to derive n from nP.

Order: The order of the finite field that defines the elliptic curve.

Characteristic: The total number of points on that curve given an elliptic curve's functions and its order. If the characteristic is q, then for any point P on that curve, qP equals P.

3.3 Commitments, Hashes, Vector Commitments

Commitment: A cryptographic primitive providing:

- **Binding**: Cannot change the committed value
- **Hiding**: The committed value remains unknown until revealed

Includes Polynomial Commitment Schemes (PCS) such as KZG/IPA/FRI.

Hash function: A function that:

- Ensures different outputs with high probability for different inputs
- Makes it computationally infeasible to discern the input from its output

- Always produces the same output for the same input
- Provides preimage/second-preimage/collision resistance
- Uses domain separation for transcripts

3.4 Arithmetic Circuit

A circuit that performs arithmetic operations on the input (not a digital circuit). The operation receiving each input is called a **gate**, and in the context of ZKP, this operation's equation becomes a **constraint**. In many NIZK systems, there's a strong positive correlation between the number of constraints and the proof generation time.

3.5 Polynomial Problem

The problem of whether one knows a polynomial that meets certain conditions. In many cases of ZKP, knowing certain input eventually gets converted or reduced to a polynomial problem.

3.6 Witness

A value needed for proof that can be obtained during the calculation process when input into a circuit. The witness represents the private inputs and intermediate values.

3.7 Instance

The public dataset where the witness is actually substituted for the gate, representing the public inputs and outputs of the computation.

3.8 Fiat-Shamir Transformation

A method to convert interactive ZKP into a non-interactive format using a random oracle (typically implemented with a hash function). The prover side can produce a value that they cannot control during the computation process, allowing the verifier side's random oracle role to be transferred to the prover side.

---

4. Abbreviations and Symbols

AIR — Algebraic Intermediate Representation
CCS — Customizable Constraint Systems
CRS — Common Reference String
DA — Data Availability
FFT — Fast Fourier Transform
FRI — Fast Reed-Solomon Interactive Oracle Proof
IPA — Inner Product Argument
IVC — Incrementally Verifiable Computation
KZG — Kate-Zaverucha-Goldberg (polynomial commitment)

MSM — Multi-Scalar Multiplication
NIZK — Non-Interactive Zero-Knowledge
PCS — Polynomial Commitment Scheme
QAP — Quadratic Arithmetic Program
R1CS — Rank-1 Constraint System
ROM — Random Oracle Model
SRS — Structured Reference String
STARK — Scalable Transparent ARgument of Knowledge
zkEVM — Zero-Knowledge Ethereum Virtual Machine
zkML — Zero-Knowledge Machine Learning
zkRollup — Zero-Knowledge Rollup
ZKP — Zero-Knowledge Proof
zkSNARK — Zero-Knowledge Succinct Non-interactive ARgument of Knowledge
zkVM — Zero-Knowledge Virtual Machine

---

5. Definition and History of ZKP

5.1 Origins

The invention and explicit mention of Zero-Knowledge Proof (ZKP) was first demonstrated in 1985 in "The Knowledge Complexity of Interactive Proof Systems" by Shafi Goldwasser, Silvio Micali, and Charles Rackoff. This foundational paper established the theoretical framework for protocols where one party can prove knowledge without revealing the knowledge itself.

5.2 Evolution to Practical Systems

1985-2012: Theoretical development period, establishing foundations but lacking practical implementations.

2013: Groth16 and earlier SNARK systems begin practical deployment, starting with ZCash development.

2016: ZCash launches as the first major production deployment of zkSNARKs (using Groth16), enabling completely private transactions on a public blockchain.

2018: Vitalik Buterin conceives zkRollup concept for Ethereum scaling. Barry Whitehat implements MVP of Plasma with zkp (Plasma snap).

2019: Loopring launches the first DEX using zkRollup technology.

2020: zkSync launches zkSync v1, a remittance network using zkSNARK validity proofs.

2021: StarkWare introduces Cairo and production STARKs. Nova folding scheme proposed.

2023: Multiple zkEVMs launch (Polygon zkEVM, Scroll, zkSync Era), offering Ethereum-compatible scaling with different levels of EVM equivalence.

2024-2025: Maturation of zkVMs, folding schemes (SuperNova, HyperNova, MicroNova), and Plonky3. EIP-4844 (Proto-Danksharding) activates, dramatically improving zkRollup economics.

---

6. ZKP Technology Architecture

6.1 Two-Layer Architecture

Modern ZKP systems are typically organized across two layers:

Frontend (Arithmetization): Converts computational statements into mathematical constraints

- Defines how computations are represented
- Determines constraint format and structure
- Affects circuit design and optimization strategies

Backend (Proving System): Generates and verifies proofs from those constraints

- Implements the cryptographic protocol
- Determines proof size, generation time, and verification cost
- Affects security assumptions and setup requirements

6.2 Key Properties

Different ZKP systems optimize for different properties:

Proof Size: From ~128 bytes (Groth16) to 100-250 KB (typical STARK)

Prover Time: Computational cost to generate proof, typically O(n log n) or better

Verifier Time: Cost to verify proof, ranging from constant (pairings) to logarithmic (FRI queries)

Setup Requirements: Trusted setup, universal setup, or transparent (no setup)

Post-Quantum Security: Hash-based systems (STARK/FRI) offer quantum resistance; pairing-based systems do not

Recursion Support: Ability to efficiently prove statements about other proofs

---

7. Frontend Systems: Arithmetization

7.1 R1CS (Rank-1 Constraint System)

Structure: Constraints of the form a × b = c where a, b, c are linear combinations of variables.

Characteristics:

- Simple and well-understood
- Foundational format for early SNARKs (Groth16, original ZCash)
- Each constraint represents one multiplication gate
- Addition and linear operations are "free"

Limitations:

- Bit operations costly (require many constraints)
- Limited expressiveness compared to modern systems
- No native lookup support

Use Cases: Early SNARK systems, simple circuits where tooling maturity matters more than efficiency.

7.2 Plonkish Gates

Structure: Polynomial constraints with custom gates and lookup tables.

Characteristics:

- Custom gates can encode multiple operations efficiently
- Native lookup table support (PlonKup, Halo2)
- More flexible than R1CS while maintaining efficient proving
- Supports range checks, bit decomposition, and table lookups efficiently

Advantages:

- Efficient hashing (can use lookup tables for S-boxes)
- Better performance for real-world applications
- Universal setup possible (reusable across different circuits)

Complexity: Higher design complexity; requires understanding of polynomial constraints.

Use Cases: Modern SNARKs (PLONK, Halo2), applications requiring efficient hashing or range proofs.

7.3 AIR (Algebraic Intermediate Representation)

Structure: Time-step state transition constraints for STARKs.

Characteristics:

- Optimized for sequential computation traces
- Natural fit for VM execution
- Constraints verify state transitions at each time step
- Execution trace represented as polynomials over time

Advantages:

- Excellent for VM traces and sequential programs
- Scales efficiently with trace length
- Natural representation for iterative computations

Use Cases: STARK-based systems, zkVMs, Cairo, long computational traces.

7.4 CCS (Customizable Constraint Systems)

Structure: Unified framework supporting multiple constraint formats.

Characteristics:

- Generalizes R1CS, Plonkish, and other systems
- Designed for folding schemes and IVC
- Flexible constraint representation

Use Cases: Nova, HyperNova, and other folding-based IVC systems.

---

8. Backend Systems: Proving Mechanisms

8.1 Pairing-Based Systems (Groth16, PLONK)

Mechanism: Uses elliptic curve pairings for succinct verification.

Setup: Requires trusted setup ceremony:

- Circuit-specific (Groth16): New setup for each circuit
- Universal (PLONK): Single setup works for all circuits up to a size

Proof Size: Smallest available (~128-192 bytes for Groth16)

Verification: Constant time (2-3 pairings), efficient on-chain

Security: Based on discrete log assumptions; not post-quantum secure

Prover Cost: FFT and MSM operations dominate; O(n log n) typical

Best For: On-chain verification, applications requiring minimal proof size, mature production deployments.

8.2 IPA (Inner Product Argument / Bulletproofs)

Mechanism: Logarithmic-size proofs via inner product arguments.

Setup: Transparent or universal (no trusted setup)

Proof Size: O(log n) group elements

Verification: O(n) but can be batched; MSM-heavy

Security: Discrete log assumption; not post-quantum secure

Recursion: Excellent recursion properties (Halo, Halo2)

Best For: Recursive proofs, applications avoiding trusted setup, moderate proof sizes acceptable.

8.3 FRI / STARK

Mechanism: Fast Reed-Solomon Interactive Oracle Proof; hash-based polynomial commitment.

Setup: Completely transparent (no setup required)

Proof Size: Larger (100-250 KB typical)

Verification: Logarithmic number of queries; hash operations

Security: Post-quantum secure (based on collision-resistant hashing and low-degree testing)

Prover: Scales well for very large computations; highly parallelizable

Best For: Large computational traces, zkVMs, post-quantum security requirements, transparent systems.

8.4 Folding / IVC (Nova, HyperNova)

Mechanism: Incrementally verifiable computation through folding schemes.

Concept: Instead of proving each step separately, "fold" multiple proof instances into one.

Characteristics:

- Low per-step prover cost
- Aggregated proof verification
- Natural for streaming or long-running computations

- Supports recursive composition

Variants:

- **Nova**: Original folding for R1CS
- **SuperNova**: Multiple circuits (non-uniform IVC)
- **HyperNova**: CCS-based with sumcheck protocol
- **MicroNova**: Extremely small proofs for sequential computation

Best For: Incremental computation, proofs of correctness for long-running programs, proof aggregation.

---

9. Polynomial Commitment Schemes

9.1 KZG (Kate-Zaverucha-Goldberg)

Mechanism: Pairing-based polynomial commitment with constant-size proofs.

Properties:

- Commitment: $c = g^{\varphi(\tau)}$ for polynomial φ
- Opening: Prove $\varphi(a) = y$ with $\pi = g^{q(\tau)}$ where $q(X) = (\varphi(X) - y)/(X - a)$
- Verification: Single pairing check

Setup: Trusted setup required; universal (powers of τ up to degree bound)

Proof Size: Single group element (~48 bytes on BLS12-381)

Verification: Constant time (one pairing)

Use Cases: PLONK, modern SNARKs, EIP-4844 KZG commitments for blob data.

9.2 IPA (Inner Product Argument)

Mechanism: Logarithmic-size commitments via recursive halving.

Properties:

- No trusted setup
- Proof size $O(\log n)$
- Verification cost $O(n)$ or $O(\log n)$ with preprocessing

Use Cases: Bulletproofs, Halo, Halo2.

9.3 FRI

Mechanism: Hash-based low-degree testing.

Properties:

- Transparent (no setup)
- Post-quantum secure
- Larger proof size
- Query-based verification

Use Cases: STARKs, Cairo, hash-based zkVMs.

---

10. Recursion and Folding

10.1 Recursive Proof Composition

Concept: Proving statements about other proofs—a proof verifies another proof's correctness.

Applications:

- Proof aggregation (many proofs → one proof)
- Unbounded computation (streaming proofs)
- Cross-system bridges (prove STARK inside SNARK)

Challenges:

- Non-native field arithmetic (proving over different curves)
- Circuit size for verifier
- Cycle of curves (Halo approach)

10.2 Folding Schemes (Nova Family)

Nova (2021):

- Fold two R1CS instances into one
- Low per-step cost
- Proof-carrying data (PCD) support

SuperNova (2022):

- Multiple circuit support (non-uniform IVC)
- Prove execution of different programs

HyperNova (2023):

- CCS-based (more expressive than R1CS)
- Sumcheck protocol integration
- Better for complex constraint systems

MicroNova (2024):

- Extremely small proofs for sequential computation
- Optimized for specific use cases

Use Cases: Long-running computations, zkVMs, proof aggregation, incremental verification.

---

11. Applications

11.1 Privacy-Preserving Transactions

ZCash: Uses Groth16 to completely conceal:

- Sender address
- Receiver address
- Transfer amount

While maintaining verifiability of:

- No double-spending
- Conservation of value
- Proper authorization

Tornado Cash: Privacy mixer using zkSNARKs for:

- Breaking on-chain transaction graphs
- Proving withdrawal rights without revealing deposit
- Selective disclosure capabilities

11.2 zkRollups

Definition: Layer 2 scaling solution where:

- Transactions executed off-chain
- State transitions proven with validity proofs (zkSNARKs/STARKs)
- Proof and minimal data posted to Layer 1
- Inherits Layer 1 security guarantees

History: Conceived by Vitalik Buterin in 2018. Barry Whitehat performed MVP implementation (Plasma snap). Loopring launched first production DEX (2019). zkSync v1 launched (2020). Multiple zkEVMs launched (2023).

Classification by State Management:

1. **Stateful zkRollup**: Manages state commitments on Layer 1 smart contracts
    - Examples: Starknet, zkSync Era, Polygon zkEVM, Scroll, Taiko, Loopring

2. **Succinct zkRollup**: Anyone can calculate state from Layer 1 data
    - Example: Sovereign Rollup
3. **Stateless zkRollup**: Users compute only their own state
    - Example: Intmax

Classification by zkEVM Type:

- **Type 1**: Exactly equivalent to Ethereum Network
    - Example: Taiko
    - Perfect compatibility, slowest proving
- **Type 2**: Fully EVM bytecode compatible
    - Example: Scroll
    - Minor modifications for proving efficiency
- **Type 3**: Mostly EVM compatible
    - Example: Polygon zkEVM
    - Some Solidity code portable
- **Type 4**: Custom VM, limited EVM compatibility
    - Examples: StarkNet (Cairo), zkSync (custom VM)
    - Best performance, requires code porting

11.3 Related zkp Solutions (Not zkRollups)

zkPorter (zkSync proposal):

- Sacrifices some zkRollup security for scalability
- Offloads DA cost to token-carrying network
- 2/3 token holder coalition could freeze (but not confiscate) assets
- Does not fully inherit Layer 1 security

Validium (zkSync):

- Trusts validator group for DA
- No Layer 1 DA cost
- Lower security than zkRollup

Volition (Starkware):

- User-selectable security/cost trade-off
- Choose DA storage trustlessness level
- Balance between price and security

11.4 Identity and Credentials

Selective Disclosure: Prove attributes without revealing full credentials:

- Age verification without birthdate
- Citizenship without passport details
- Credentials without identity linkage

Applications:

- Privacy-preserving KYC
- Anonymous voting with eligibility proof
- Regulatory compliance with minimal disclosure

11.5 Verifiable Machine Learning (zkML)

Capabilities:

- Prove model trained on specific dataset
- Verify inference ran with specific weights
- Demonstrate model properties without revealing model

Use Cases:

- Trustworthy AI attestation
- Private inference
- Model provenance
- Regulatory compliance for AI

11.6 Cross-Chain Bridges

ZK-enabled bridges:

- Prove state on source chain
- Verify proof on destination chain
- Eliminate trusted intermediaries

Advantages:

- Trustless cross-chain transfers
- No validator set reliance
- Cryptographic security guarantees

---

12. Programming Languages and Libraries

12.1 Domain-Specific Languages

Circom (iden3):

- JavaScript-like syntax
- R1CS circuit generation
- Mature toolchain and ecosystem
- Used in production (Tornado Cash, Hermez)

Cairo (StarkWare):

- Designed for STARKs
- AIR-based constraints
- Turing-complete STARK-friendly CPU architecture
- Powers Starknet

Noir (Aztec):

- Rust-like syntax
- Multiple backend support
- Focus on developer experience
- Universal verifier approach

Leo (Aleo):

- Type-safe functional language
- SNARK-optimized
- Integrated with Aleo platform

12.2 Libraries and Frameworks

Bellman (Rust):

- Groth16 implementation
- Used in ZCash
- Mature and audited

Halo2 (Rust):

- PLONK-based with lookup arguments
- No trusted setup (IPA)
- Excellent recursion
- Used in ZCash Orchard, Scroll

Plonky2/Plonky3 (Rust):

- Fast PLONK implementation
- FRI-based (Plonky2) or flexible PCS (Plonky3)
- 2M Poseidon2 hashes/s on M3 Max

- Powers SP1, Valida

arkworks (Rust):

- Comprehensive cryptographic primitives
- Multiple curve implementations
- Modular proving system components

gnark (Go):

- Go-based SNARK library
- Groth16, PLONK support
- Production-ready

Winterfell (Rust):

- STARK prover and verifier
- AIR-based
- Used in Polygon Miden

12.3 zkVMs

SP1 (Succinct Labs):

- RISC-V zkVM
- Plonky3-based
- Supports Rust programs

Risc Zero:

- RISC-V zkVM
- STARK-based
- Mature tooling

zkMIPS:

- MIPS architecture zkVM
- EVM compatibility focus

Valida:

- Custom ISA zkVM
- Plonky3-based
- Performance-optimized

---

13. Performance Trade-offs

13.1 Fundamental Trade-offs

Proof Size vs. Prover Time:

- Groth16: Tiny proofs (~128 B), slow proving, requires trusted setup
- STARK: Large proofs (100-250 KB), fast proving at scale, transparent

Verification Cost vs. Proof Size:

- Pairing-based: Constant verification, smallest proofs, on-chain friendly
- FRI-based: Logarithmic queries, larger proofs, post-quantum secure

Setup Requirements vs. Flexibility:

- Circuit-specific setup: Best performance, new ceremony per circuit
- Universal setup: One ceremony for all circuits, slightly larger proofs
- Transparent: No setup, larger proofs or higher verification cost

13.2 Recursion Capabilities

Native recursion (Halo2, Plonky2):

- Efficient proof-of-proof verification
- Enables proof aggregation
- Critical for zkVMs and unbounded computation

Cross-system recursion:

- Prove STARK inside SNARK (on-chain verification)
- Requires non-native field arithmetic
- Higher overhead but enables best-of-both-worlds

13.3 Practical Considerations

Hardware acceleration:

- GPU acceleration for MSM operations
- ASIC development for high-throughput
- Critical for production zkRollup sequencers

Parallelization:

- STARKs: Highly parallelizable
- Folding schemes: Sequential by nature
- Circuit partitioning strategies

Memory requirements:

- Large circuits require significant RAM
- FFT operations memory-intensive
- Proof aggregation reduces peak memory

---

14. Security Considerations

14.1 Trusted Setup

Circuit-Specific Setup (Groth16):

- New ceremony required for each circuit
- "Toxic waste" (secret randomness) must be destroyed
- If τ is recovered, soundness breaks (false proofs accepted)
- Multi-party computation (MPC) ceremonies mitigate risk

Universal Setup (PLONK, Halo2):

- Single ceremony for all circuits up to size bound
- Reusable across different applications
- Significantly improves practical security
- Still requires secure ceremony execution

Transparent Systems (STARK, Bulletproofs):

- No setup required
- Eliminates trusted setup risk entirely
- Trade-off: Typically larger proofs

14.2 Parameter Selection

Security levels:

- Curve choice affects security (BN254 ~100-128 bit, BLS12-381 ~128 bit)
- Field size must exceed security parameter
- Binding factor selection in FRI
- Soundness error must be negligible

Common mistakes:

- Insufficient constraint randomization
- Weak Fiat-Shamir hash function
- Improper domain separation
- Inadequate field size for operations

14.3 Circuit Design Vulnerabilities

Under-constrained circuits:

- Missing constraints allow invalid proofs
- Similar to smart contract logic bugs
- Requires careful auditing

Arithmetic errors:

- Overflow/underflow in finite fields
- Division by zero handling
- Range constraint violations

Nondeterminism:

- Witnesses must be deterministic from public inputs
- Prover-chosen values must be properly constrained

14.4 Implementation Vulnerabilities

Historical vulnerabilities:

- **Frozen Heart (Bulletproofs)**: Soundness break via malformed proof
- **Groth16 verification**: Incorrect pairing check implementation
- **zkSNARK setup**: Parameter generation issues

Common issues:

- Fiat-Shamir transcript errors
- Incorrect subgroup checks
- Malleability attacks
- Memory safety in proof generation

14.5 Audit Requirements

Circuit audits:

- Constraint completeness verification
- Arithmetic correctness
- Witness generation review

Cryptographic audits:

- Parameter selection validation
- Setup ceremony review (if applicable)
- Implementation correctness

Best practices:

- Formal verification where possible
- Extensive testing with edge cases
- Independent security reviews
- Bug bounty programs

---

15. Mathematical Derivations

15.1 Groth16 via QAP (Quadratic Arithmetic Program)

Setup: Given R1CS constraints, construct polynomials:

- Let $A_w(X) = \sum w_i A_i(X)$
- Let $B_w(X) = \sum w_i B_i(X)$
- Let $C_w(X) = \sum w_i C_i(X)$
- Let $t(X) = \prod(X - r_j)$ be the target polynomial

Requirement: $A_w(\tau)\cdot B_w(\tau) - C_w(\tau) = h(\tau)\cdot t(\tau)$

This means the polynomial on the left vanishes at all roots of $t(X)$, which happens if and only if all constraints are satisfied.

Proof Structure = $(A\in G1, B\in G2, C\in G1)$

Verification: Three pairing checks confirm the QAP relationship holds.

15.2 KZG Evaluation Proof

Commitment: $c = g^{\varphi(\tau)}$ for polynomial φ

Opening at point a with claimed value $y = \varphi(a)$:

1. Compute quotient $q(X) = (\varphi(X) - y)/(X - a)$
2. Generate proof $\pi = g^{q(\tau)}$
3. Verification: $e(c/g^y, g) = e(\pi, g^{\tau}/g^{a})$

Result: Constant-size proof (one group element) proves polynomial evaluation.

15.3 Sumcheck Protocol (Sketch)

Goal: Prove $S = \sum_{x\in\{0,1\}^n} g(x)$ for multivariate polynomial g

Method:

1. Prover sends univariate $h_1(X_1) = \sum_{x_2,...,x_n} g(X_1, x_2, ..., x_n)$
2. Verifier checks $h_1(0) + h_1(1) = S$ and sends random r_1
3. Recurse with $g'(x_2, ..., x_n) = g(r_1, x_2, ..., x_n)$ and claim $S' = h_1(r_1)$
4. After n rounds, verifier checks final evaluation directly

Result: Reduces sum verification to polynomial evaluation, logarithmic communication.

15.4 FRI Low-Degree Test

Goal: Prove committed polynomial has degree ≤ d

Method:

1. Commit to polynomial evaluations on domain

2. Apply random linear combination and domain reduction
3. Repeat folding with fresh randomness
4. Final polynomial is constant (degree 0)
5. Verification: Query random positions, check consistency

Result: Logarithmic rounds, query-based verification, post-quantum secure.

---

16. Measurements and Representative Sizes

16.1 Proof Sizes (Typical Configurations)

System	Curve / Security	Proof contents	Compressed size	Uncompressed size (on-chain)
Groth16	BN254 (~100–128-bit)	A∈G1, B∈G2, C∈G1	~128 B (32+64+32)	≈256 B (64+128+64)
Groth16	BLS12-381 (128-bit+)	A∈G1, B∈G2, C∈G1	~192 B (48+96+48)	≈384 B (96+192+96)
KZG eval proof	BLS12-381	π∈G1	48 B	96 B (uncompressed)
IPA PC (poly eval)	Curve-group	O(log n) elements	logarithmic	logarithmic
STARK/FRI	Hash-based	Queries + commitments	100–250 KB (typical)	same

16.2 EIP-4844 Blob Economics

Parameter	Symbol	Value
Bytes per field element	BYTES_PER_FIELD_ELEMENT	32
Field elements per blob	FIELD_ELEMENTS_PER_BLOB	4096
Bytes per blob	—	131,072 bytes

Blob gas per blob	GAS_PER_BLOB	2^17 = 131,072
Target / Max blobs per block	—	3 target / 6 max

Cost calculation: cost per blob = base_fee_per_blob_gas × GAS_PER_BLOB

Example: At 1 gwei per blob gas ⇒ 131,072 gwei per blob (≈1.31e-4 ETH)

Significance: EIP-4844 (activated 13-Mar-2024) dramatically reduced zkRollup data posting costs by introducing dedicated blob storage with separate fee market.

---

17. Choosing a Stack

For minimal proof size & fast on-chain verification:

- Pairing/KZG-based (Groth16/PLONK)
- Mature Solidity verifiers
- Best for frequent on-chain verification

For transparent systems & long traces:

- FRI/STARK
- No trusted setup
- Post-quantum security
- Best for zkVMs and large computations

For flexible recursion/folding:

- Halo2, Plonky2/3, Nova/HyperNova
- Efficient proof composition
- Best for incremental computation

For zkRollups:

- Type 1-3 zkEVM: Groth16 or PLONK with KZG
- Type 4: STARK-based (Cairo for Starknet)
- Consider blob availability post-EIP-4844

---

18. Ecosystem Snapshot (2024-2025)

Plonky3: >2M Poseidon2 hashes/s on M3 Max (reported); powering SP1/Valida

Nova/HyperNova/MicroNova: Advancing folding/IVC with on-chain verification improvements

zkVMs: Maturing rapidly; universal verifiers and aggregation becoming standard

EIP-4844: Proto-Danksharding live since March 2024; blob fee market significantly improves zkRollup economics

zkEVM Competition: Multiple implementations (Polygon zkEVM, zkSync Era, Scroll, Taiko) racing toward full EVM equivalence

Post-Quantum Transition: Growing attention to FRI-based systems and quantum resistance roadmaps

---

19. References

Foundational Papers

[1] Goldwasser, S., Micali, S., & Rackoff, C. (1985). "The Knowledge Complexity of Interactive Proof Systems." *STOC 1985*.

[2] Blum, M., Feldman, P., & Micali, S. (1988). "Non-Interactive Zero-Knowledge and Its Applications." *STOC 1988*.

[3] Groth, J. (2016). "On the Size of Pairing-based Non-interactive Arguments." *EUROCRYPT 2016* (ePrint 2016/260).

[4] Ben-Sasson, E., Bentov, I., Horesh, Y., & Riabzev, M. (2018). "Scalable, transparent, and post-quantum secure computational integrity." *IACR ePrint 2018/046*.

Modern Systems

[5] Gabizon, A., Williamson, Z.J., & Ciobotaru, O. (2019). "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge." *IACR ePrint 2019/953*.

[6] Bowe, S., Grigg, J., & Hopwood, D. (2019). "Recursive Proof Composition without a Trusted Setup." *IACR ePrint 2019/1021* (Halo).

[7] Kothapalli, A., Setty, S., & Tzialla, I. (2021). "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes." *IACR ePrint 2021/370*.

[8] Kothapalli, A., & Setty, S. (2022). "SuperNova: Proving universal machine executions without universal circuits." *IACR ePrint 2022/1758*.

[9] Kothapalli, A., Setty, S., & Tzialla, I. (2023). "HyperNova: Recursive arguments for customizable constraint systems." *IACR ePrint 2023/573*.

[10] Boyle, E., et al. (2024). "MicroNova: Extremely Small Proofs for Sequential Computation." *IACR ePrint 2024/2099*.

[11] Polygon Labs (2024). "Plonky3: The Next Generation of ZK Proving Systems."

Polynomial Commitments

[12] Kate, A., Zaverucha, G.M., & Goldberg, I. (2010). "Constant-Size Commitments to Polynomials and Their Applications." *ASIACRYPT*.

[13] Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., & Maxwell, G. (2018). "Bulletproofs: Short Proofs for Confidential Transactions and More." *IEEE S&P*.

zkRollups and Scaling

[14] Buterin, V. (2018). "On-chain scaling to potentially ~500 tx/sec through mass tx validation." *Ethereum Research*.

[15] Ethereum Foundation (2024). "EIP-4844: Shard Blob Transactions (Proto-Danksharding)."

[16] Ethereum.org (2024). "Dencun Upgrade FAQ" (activation 13-Mar-2024; blob lifetime).

[17] Blocknative (2024). "Blob congestion event analysis; still cheaper than calldata in practice."

Security

[18] Trail of Bits (2022). "The Frozen Heart: ZK Proof Vulnerability." *Blog post on Bulletproofs vulnerability*.

[19] OpenZeppelin (2023). "Auditing Zero-Knowledge Circuits."

[20] Consensys Diligence (2024). "zkSNARK Security Best Practices."

Online Resources

[21] ZK Whiteboard Sessions. https://www.youtube.com/@zk-learning

[22] IACR ePrint Archive. https://eprint.iacr.org/

[23] Ethereum Research Forum. https://ethresear.ch/

[24] ZKProof Standards. https://zkproof.org/

[25] BGIN Resources. https://bgin-global.org/

---

20. Appendices

Appendix A: Glossary

AGM — Algebraic Group Model
AIR — Algebraic Intermediate Representation
CCS — Customizable Constraint Systems
CRS — Common Reference String
DA — Data Availability
FFT — Fast Fourier Transform
FRI — Fast Reed-Solomon Interactive Oracle Proof
IPA — Inner Product Argument
IVC — Incrementally Verifiable Computation
KZG — Kate-Zaverucha-Goldberg polynomial commitment
MSM — Multi-Scalar Multiplication
NIZK — Non-Interactive Zero-Knowledge
PCS — Polynomial Commitment Scheme
QAP — Quadratic Arithmetic Program
R1CS — Rank-1 Constraint System
ROM — Random Oracle Model
SRS — Structured Reference String
STARK — Scalable Transparent ARgument of Knowledge
zkEVM — Zero-Knowledge Ethereum Virtual Machine
zkML — Zero-Knowledge Machine Learning
zkRollup — Zero-Knowledge Rollup
ZKP — Zero-Knowledge Proof
zkSNARK — Zero-Knowledge Succinct Non-interactive ARgument of Knowledge
zkVM — Zero-Knowledge Virtual Machine

Appendix B: System Comparison Tables

Backend System Comparison

Family	Setup	Assumptions	Proof size	Verify time	Prover cost	Notes
Pairing/KZG	SRS (circuit/universal)	AGM/ROM, DLOG	Smallest	Few pairings	FFT+MSM heavy	Mature on-chain
IPA	Transparent/universal	DLOG	O(log n)	MSMs	Moderate–High	Good recursion
FRI/STARK	Transparent	Hash/LDT	100–250 KB+	Queries	Scales well	VM traces

Folding/IVC	Depends on PCS	Sumcheck + PCS	Aggregated	Succinct	Low per-step	Streaming/PCD

Frontend System Comparison

Front-End	Primary use	Lookups	Trace style	Strengths	Considerations
R1CS	Early SNARKs	Limited	Gate constraints	Simple	Bit ops costly
Plonkish	SNARKs w/ custom gates	Native	Polynomial constraints	Efficient hashing/lookups	Design complexity
AIR	STARK systems	Common	Time-step transitions	VM traces	Larger proofs
CCS	Unified	Via gadgets	Flexible	IVC-friendly	Advanced tooling

Appendix C: On-Chain Verification Notes

Groth16 on BN254:

- Mature Solidity verifiers available
- Uses pairing precompiles (EIP-197)
- ~300k gas typical verification cost
- Most widely deployed for on-chain verification

KZG on BLS12-381:

- Point-evaluation precompile (EIP-4844)
- Designed for blob verification
- Can be used for general polynomial commitments
- ~50k gas for single evaluation proof

IPA verifiers:

- Cost dominated by multi-scalar multiplications (MSMs)
- Batch openings strongly recommended
- Higher gas cost than pairing-based
- Benefits from EIP-2537 (BLS12-381 operations)

Appendix D: zkRollup Considerations

Decentralization of Block Producers: Many zkRollups have not yet fully decentralized block producers due to:

- High computational cost of proof generation
- Risk of bugs in new systems
- While this doesn't compromise security (inherited from Layer 1), it raises concerns about service continuity

Proto-danksharding/Danksharding Security Implications: Time-limited data (e.g., one month via proto-danksharding) represents a novel approach:

- Enables 1/N security assumption (one honest node prevents DA attack)
- When writing to Layer 1 calldata: N ≈ 5000 (Ethereum node count)
- When using proto-danksharding: N ≈ 20-100 (rollup node count)
- Major Layer 2s can maintain large N
- Smaller "Rollup-as-a-Service" deployments may have limited N

zkRollup ≠ Zero-Knowledge?: Most zkRollups are not confidential. The "zero-knowledge" property is used primarily for:

- Hiding intermediate computation details for efficiency
- Not for privacy (data is public)
- Some argue "validity rollup" is more accurate terminology

Does zkRollup = Bridge?: Tokens natively issued on Layer 2 that are not bridged from Layer 1 may not fully inherit Layer 1 security if they conduct independent governance or forks. However, this mirrors existing Layer 1 tokens with governance or limited liquidity. Currently considered a non-critical concern.

Disclaimer: The views expressed in this document are personal views of the participating members of the BGIN community and should not be seen as the official views or recommendations of the institutions with which they are affiliated.

The technology described in this document was made available from contributions from various sources, including members of BGIN and others. Although BGIN has taken steps to help ensure that the technology is available for distribution, it takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this document or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any independent effort to identify any such rights.

END OF DOCUMENT

BGIN SR 0011 – Zero-Knowledge Proofs (ZKPs): Technology and Applications Study Report

Version: 1.0 Consolidated
Date: 2025
© 2025 BGIN (Blockchain Governance Initiative Network)
All Rights Reserved

Appendix Z

BGIN SR 011 — Study Report
Zero-Knowledge Proofs (ZKPs) and Their Applications (compressed draft version)

Author: Leona Hioki (BGIN) and contributors

Editor: Mitchell Travers (BGIN)

Editorial policy: This edition expands all topics, corrects inaccuracies, and appends the full original draft as Appendix Z (verbatim archive) so there is no omission.

BGIN SR 011 (Draft)

(Study Report)

ZKP and its application

Part1: ZKP Technology

Leona Hioki BGIN
Contributer
28.07.202

v0.0.1

Introduction

Part1: ZKP Technology -

-Definition of ZKP:
ZKP,
NIZK
- Terminology
- Composition and Characteristics of NIZK
- Recursive ZKP

- Classification of NIZK:
-History of NIZK:
-Backend:
- Frontend
- Classification

Part2: ZKP and Society
- Programming Languages and Libraries Exclusive to ZKP:
- Trade-offs in ZKP Performance
- History of Vulnerability Discoveries in ZKP
- Examples of ZKP Applications

- zkRollup: Definition of zkRollup,
-History of zkRollup
-Concerns about zkRollup

Disclaimer

Researcher map

Table of contents

Executive Summary

This report consolidates modern ZK practice: arithmetization choices (R1CS, Plonkish, AIR, CCS), proving back-ends (pairing/KZG, IPA, FRI/STARK, sumcheck-centric), recursion and folding/IVC (Nova/SuperNova/HyperNova), zkVMs, and post-EIP-4844 rollup economics. It adds concrete derivations (Groth16/QAP, KZG, sumcheck, FRI), proof size and cost estimates, and security/audit checklists suitable for production deployments.

1. Scope and Audience

Scope: practical non-interactive proofs in production—zkSNARKs, zkSTARKs, and folding-based IVC—and their use in privacy, scalability, and proofs of computation. Audience: protocol engineers, auditors, and product teams.

2. Conventions and Notation

We write fields as $\mathbb{F}_q$ ($q=p^k$), elliptic-curve groups as G1/G2/G_T, and use CRS/SRS for reference strings.

3. Terms and Definitions (Corrected & Expanded)

3.1 Zero-Knowledge Proof (ZKP) & NIZK

ZKP: completeness, soundness (or knowledge soundness for SNARKs), and zero-knowledge. NIZK is obtained natively or via Fiat–Shamir (ROM).

3.2 Finite Fields, Groups, Curves

- Finite field $\mathbb{F}_q$ has $q=p^k$; characteristic($\mathbb{F}_q$)=p; every non-zero element has an inverse.
- Elliptic curves over $\mathbb{F}_q$ form abelian groups with identity $\mathcal{O}$; for finite group order n, $n \cdot P=\mathcal{O}$ for any point P.
- Group order ≠ field characteristic.

3.3 Commitments, Hashes, Vector Commitments

- Commitment: binding + hiding. Includes PCS (KZG/IPA/FRI).

Hash: preimage/second-preimage/collision resistance; domain separation for transcripts.

3.4
Elliptic Curve

equation that can be represented as y^2 = x^3 + x +

Note to entry: In this context, an elliptic curve refers to one over a finite field. Points P and Q on the elliptic curve have a defined addition operation, making P + Q = R. While direct multiplication, such as P x Q, isn't defined, scalar multiplication like P+P=2P and nP (n times P) is defined. Due to the discrete logarithm problem, it's challenging to derive n from nP.

3.5

order

order of the finite field that defines the elliptic curve

3.6

characteristic

total number of points on that curve given an elliptic curve's functions and its order

Note to entry: If the characteristic is q, then for any point P on that curve, qP equals P.

3.7
hash function

function that ensures different outputs with high probability for different inputs, is challenging to discern the input from its output, and always produces the same output for the same input

3.8
bit Commitment

verification system that guarantees the committed value remains unchanged and remains unknown (hiding) until it's revealed. It can be described that most of today's blockchains are this system with a timestamp added

3.9

hiding

state of commitment without revealing

Note to entry: If various verifications can be done without revealing, it can be applied to confidentiality.

3.10

arithmetic circuit

circuit that performs arithmetic operations on the input, not a digital circuit

Note to entry: The operation receiving each input is called a gate, and in the context of ZKP, this operation's equation becomes a constraint. In many NIZK systems, there's a strong positive correlation between the number of constraints and the proof time.

3.11

polynomial problem

problem of whether one knows a polynomial that meets certain conditions

Note to entry: In many cases of ZKP, knowing certain input eventually gets converted or reduced to a polynomial problem.

3.12

witness

value needed for proof that can be obtained during the calculation process when input into a circuit

3.13

instance

dataset where the witness is actually substituted for the gate

3.14

fiat-shamir transformation

method to convert interactive ZKP where a random oracle is used into a non-interactive format

Note to entry: Using a hash function, the prover side can produce a value that they cannot control during the computation process, allowing the verifier side's random oracle role to be transferred to the prover side.

4. Abbreviations and symbols

In this document, the following abbreviations and symbols are used.

IPA —

FRI —

NIZK — non-Interactive zero-knowledge

R1CS —

ZKP — zero-knowledge proof

zkSNARKs —

5 Definition of ZKP

The invention and explicit mention of Zero Knowledge Proof (ZKP) was first demonstrated in 1985 in "The Knowledge Complexity of Interactive Proof Systems" by authors Shafi Goldwasser, Silvio Micali, and Charles Rackoff.

The features generally defined for it can be described as follows:

For a given statement, within a protocol where a Prover and Verifier exist, a ZKP satisfies the following:

Completeness: When the statement is true, the verifier can be convinced of its truthfulness by the prover.

Soundness: If the statement is false, the prover cannot convince the verifier to accept it as true.

Zero-Knowledge: The verifier learns only the truth of the statement and no other information.

・ NIZK

Among ZKPs, those where the verifier and prover do not require interactive communication and it concludes solely with the verifier checking the proof sent by the prover are called Non-Interactive Zero-Knowledge (NIZK). zkSNARKs, zkSTARKs, and BulletProofs fall under this NIZK category

・ Common reference string generated from random numbers

The defining features for this can be described as:

Procedure:

・There exists a value called a common reference string (crs) that is generated from random numbers.

・From the crs, input, and witness, the prover calculates a proof.

・ The verifier either accepts or rejects the proof.

For this procedure's protocol, what satisfies the characteristics of a NIZK is the following.

Completeness:
When the proof holds true, the verifier can be assured of its accuracy by the prover.

Non-Adaptive Soundness:
If the proof isn't true, the prover cannot make the verifier accept it as such.

Adaptive Soundness:
If the proof isn't true, the prover cannot make the verifier accept it as such, even when the prover makes a proof after seeing the crs.

Non-Adaptive Zero-Knowledge:
The verifier only learns about the proof's truthfulness and no additional information.

Adaptive Zero-Knowledge:
Even the best attacker can not make any proof distinguishable from another after seeing the crs.

End of definition.

In many cases, a Non-Interactive ZKP can be obtained by applying the Fiat-Shamir transformation to an Interactive ZKP.

6.Composition and Characteristics of NIZK

NIZK (Non-Interactive Zero-Knowledge) is a protocol within ZKP (Zero-Knowledge Proof) that has the characteristic of reducing the computational cost for the verifier while increasing the computational cost for the prover. Also, the communication costs for both sides have been significantly reduced. Many developers are trying to improve the high computational cost of the prover. At the same time, many are attempting to ensure that the verification cost remains within a tolerable range, even on inherently

inefficient computational systems like blockchains.

Given these characteristics, it's a system that offers significant benefits to both verifiers and provers, which can be summarized as:

Verifier: Low computational cost, ability to verify input or properties of the prover. Prover: High computational cost, can undergo verification while avoiding disclosure of sensitive or unnecessary data.

It's evident that while the verifier gains pure efficiency, the prover benefits from privacy and efficiency through data omission.

Many methods make up the properties of zkSNARKs, a representative of NIZK. While there are many types of zkSNARK schemes, most of them commonly follow this structure:

1)Circuit: A circuit representing constraints related to the input.

2)Transformation: Converts the circuit into polynomial problems or SAT problems.

3)Proving: The prover demonstrates that they know the solution to the transformed problem.

4)Verifying: The verifier checks the proof and decides to accept or reject it.

1)and 2) are sometimes referred to as the frontend, while 3) and 4) are called the backend.

-Regarding Trusted Setup:

In some NIZKs, when generating the Proving key or Verifying key from the circuit, if one abuses the process information, there's a possibility to forge the proof and lose the zero-knowledge property of the proof. This kind of information is called "toxic waste," and sometimes circuit creators are obliged to discard this information. There are mechanisms, like ptau ceremony, to ensure that toxic waste can't be abused unless all participants collude. It's a system where multiple people produce toxic waste.

There are cases where this toxic ceremony is produced for each circuit and cases where it can be reused across various circuits worldwide, with the latter particularly called the Universal Trusted Setup. In practical applications, with mechanisms that allow this, such as PlonK, security can be significantly enhanced, leading to the state where having a Trusted Setup does not directly compromise Trustlessness.

7. Recursive zkSNARKs

In zkSNARKs, when a circuit that represents constraints can be expressed in a Turing-complete language or in a generic manner, it becomes possible to write the code for zkSNARKs verification within that circuit. This means that within a zkSNARKs proof, the verification of another zkSNARKs' Proof can be incorporated.

Such Recursive zkSNARKs are vital in many zkSNARKs applications because of improved versatility, the ability to split and parallelize proof computations, and the ability to customize data confidentiality. As a result, most of the ZKP libraries used in recent applications have this recursive feature implemented.

Previously, this recursive characteristic was seen as rare and challenging. The difficulties can be described as follows:

Consider the task of performing the computation for zkSNARKs verification, specifically the pairing computation, within a circuit. The pairing computation itself is a calculation in the context of elliptic curves. However, when executed within the circuit, the calculation is done within the bounds defined by the elliptic curve. Just like how 9 behaves differently in F5 compared to F7, calculations might not match due to discrepancies, causing a loss of information and making verification impossible.

Given this, it might seem preferable at first to compute the pairing within a ZKP circuit using elliptic curves that have matching orders. But this approach results in curves that are vulnerable to the SSSA attack. A solution was to create two sets of elliptic curves. The first set, E1, has an order of p and a characteristic of q. The second set, E2, has an order of q and a characteristic of p. The challenge was to find a pair in which the order from E1 would match the characteristic from E2, and vice versa. This is where the Pasta Curve comes into play. Even though the order and characteristic aren't identical, by using the alternating nature of these numbers, it became possible to achieve recursion without repetition.

In contrast, zkSTARKs doesn't rely on elliptic curves, so there's no risk of the SSSA attack. This makes it feasible to construct a Recursive ZKP that can undergo any number of repetitions within the bounds of a single finite field.

7.1 On Cyclic Recursive ZKP

Even when performing Recursive ZKP an unlimited number of times, usually a different ZKP circuit is required for each layer, necessitating the preparation of a circuit for each of those layers. However, when the proof being verified originates from that very circuit, it is possible to generate an infinite hierarchical Recursive Proof using just a single circuit. This is called a Cyclic Recursive ZKP. In the case of a Recursive ZKP, it is customary to embed a verifying key within the circuit. However, for Cyclic Recursive ZKP, it is not possible to embed its own verifying key when creating the circuit, so this issue is addressed by verifying the identity of the circuit.

Here are the defining features of Cyclic Recursive ZKP:

A proof P, produced by a given circuit C, can be reintroduced into C to generate the subsequent proof P'.

8. Classification of NIZKs

The development of practical NIZKs, such as zkSNARKs and zkSTARKs, has been taking place over the past decade, notably beginning with the Pinocchio Protocol in 2013, and greatly propelled by Groth16 in 2016.

In the Pinocchio Protocol, constraints represented in the R1CS format were converted to QAP, achieving verifiable computing. Groth16 effectively addressed these issues, becoming, in practice, the first widely-used NIZK system.

While Groth16 utilized pairings, zkSTARKs in 2017 replaced this with FRI, leading to a significant improvement in the computation speed of proof creation. In 2019, PlonK addressed a major inefficiency in Groth16's R1CS, where the polynomial degree increased inefficiently for constraints (Copy Constraints) using the same variables. This inefficiency was dramatically optimized using a technology called coordinate accumulator. Additionally, Ultra PlonK made it possible to write constraints in higher-degree polynomials, allowing for the assembly of various free gates and custom gates.

Subsequent NIZK developments have mainly focused on the previously mentioned achievement of Recursive ZKP. Notable systems include Redshift, a combination of FRI and PlonKish, Plonky2, Halo2 which backs Ultra PlonK with IPA, and Nova, which can sum the computational processes of R1CS. Plonky2, renowned for significantly accelerating the generation speed of Recursive ZKP, achieved faster processing performance by reducing the number of finite fields in the definition body while ensuring security did not fall below 100 bits. By expanding the definition body and increasing the dimensions, it was made possible to perform individual computational costs within a smaller finite field.

8.2 Backend

On Pairing and PCS

Pairing refers to the operation between points on the elliptic curve, characterized by its bilinearity. The bilinearity can be expressed as:

For points P, P', Q, Q' on the twisted elliptic curve group concerning pairing:

(1) $en(P+P',Q') = en(P,Q)\ en(P',Q)$

(2) $en(P,Q+Q') = en(P,Q)\ en(P,Q')$

(3) $en(P,P) = 1$

(4) $en(P,Q \neq O)=1 \Rightarrow P=O$

This can verify equations containing both addition and multiplication once each, allowing calculations to be performed while data remains confidential. This meets the requirements of zkSNARKs. Specifically, in the QAP format after converting polynomial problems in zkSNARKs, it allows for the verification of polynomial equations at a single point while the polynomial remains concealed.

For $A(x)B(x)-C(x) = Z(x)H(x)$, verification is conducted at point $x=s$. In this instance, the equation can be resolved with a single multiplication, hence the use of pairing.

For more flexible polynomial equations like in PlonK, Polynomial Commitment Schemes (PCS) are employed. Unlike regular bit commitments, polynomial commitments can commit to functions, allowing for the insertion of various values to obtain flexible outputs.

About FRI

FRI is an algorithm to ascertain and verify knowledge of a polynomial. It leverages the

difficulty of interpolating a high-degree polynomial using a small number of points for those unfamiliar with the polynomial.

Example: For values between 0 and n, if a polynomial satisfies the condition P(Q(x)) = 0, then P(Q(x)) can be represented as Z(x)H(x).

To prove knowledge of such a Q(x) when P and Z are known, one needs to demonstrate: "I am aware of several points on Q and the corresponding points on H for the same x values."

However, given Z is known, for each point x=s, one can compute P(t) and divide it by Z(s). This allows even arbitrary values of t to satisfy the equation. Thus, proving these points lie on the same polynomial becomes essential. The difficulty of ensuring many sample points belong to a lower-degree polynomial unless you're aware of that polynomial forms the core of the security concept. This concept is foundational to FRI.

Among mechanisms that ensure ZKP's security, FRI is rare in its quantum resilience and does not depend on security assumptions like the discrete logarithm problem.

About the Folding Scheme

In R1CS, the expression a times b equals c is termed an instance. Combining different instances born from different witnesses within the same circuit, like through addition, is tricky. Yet, this becomes possible when applying an extended R1CS called Relaxed R1CS. This method of merging two instances into one is termed the Folding Scheme. By continually merging two into one, the final proof becomes highly compact.

About the sumcheck protocol

It's a succinct zero-knowledge proof protocol. By repeatedly substituting boolean values and random values into the target polynomial, one obtains the proof. It's employed within the Folding Scheme.

About IPA

IPA is a security mechanism for zero-knowledge proofs, reliant solely on elliptic curve point computations. Suitable for straightforward computation verifications like Bullet Proof, it's also used in blockchain…

8.3 Front-end

About R1CS

R1CS is the foundational ZKP form, where the constraints a circuit must satisfy are denoted as a * b = c.

About Plonkish gate

Plonkish denotes constraints of circuits presented in pioneering papers like PlonK or its extensions. Initially denoted as XXX in the paper, it can transition to higher-degree polynomials, and these flexible constraints for high-degree polynomials are dubbed custom gates. PlonK encompassing custom gates is commonly called Ultra PlonK.

About lookuptable

It's an Ultra PlonK feature, drawing inspiration from digital circuit lookuptables. It's essentially a truth table correlating input and output bits. Instead of conducting intricate bit computations unsuited for PlonK, it directly fetches the resultant boolean value, thereby downsizing the circuit. This is notably valuable when calculating hashes using PlonK.

About AIR

It's a format to inscribe constraints of zkSTARKs-provable polynomials.

About CCS

CCS is a constraint system crafted to simultaneously depict R1CS, Plonkish, and AIR.

8.4 Classification

Using the front-end and back-end, they can be classified as follows:

ps) Pedersen commitment

9 Tools for ZKPs

1. Library
2. Language
3. security
4. IDE
5. acceleration (GPU/ASIC)

Bellman
Bellman is an initial Rust library that made it somewhat straightforward to implement zero-knowledge proofs. With it, Groth16 circuits could be implemented. Later community forks incorporated support for PlonK, among others.

Circom
Circom is the most widely-used dedicated language for zero-knowledge proofs, capable of outputting R1CS. In tandem with snark.js, it allows for easy generation of zero-knowledge proofs. Both Groth16 and PlonK are supported. Additionally, compilations from Circom to various zero-knowledge proofs like zkSTARKs or Halo2 are separately supported by diverse communities.

Cairo
Cairo is a language dedicated to zkSTARKs and compiles to AIR.

Lurk
Lurk is the most suitable language for producing Nova outputs.

Winterfell
Winterfell is an open-source zkSTARKs library in Rust.

Solidity/zkEVM
Originally a language for smart contracts, Solidity, with the introduction of a new compilation environment, can now be translated into various zkEVM circuits for diverse zero-knowledge proofs. Translations to Halo2 and PlonK are particularly renowned.

Aleo

10 ZKP Performance

When evaluating the performance of ZKPs, it is clear to consider the following basic criteria:

1) Shortness of proof generation time relative to the number of constraints.
2) Shortness of verification time.
3) Smallness of proof data relative to the number of constraints

As a note, the proof generation time is generally constant in each ZKP and does not depend on the size of the circuit. Each of these metrics can be hard to quantify definitively since they can be parallelized and made redundant by Recursive Proofs.

Whether Recursive Proof is feasible can also affect these metrics.

With this in mind, generally speaking, if we disregard Recursion, a negative correlation can be observed between proof generation time and proof data size.

A negative correlation is also observed between the shortness of verification time and the shortness of proof generation time relative to the number of constraints. For instance, in cases like FRI, a trade-off can be manipulated with parameters. This means it's adjustable whether to burden the verification side or the proof side.

From a security standpoint, whether a trusted setup is needed, or if a global one-time trusted setup suffices, or if it's unnecessary, can also be considered a performance criterion.

11 Vulnerability of ZKP

1) Parameters

Even if the ZKP scheme itself is secure and has a security proof, depending on how the parameters are set, a vulnerable ZKP proof can emerge. There's often a significant incentive in many cases to set parameters that might decrease security for purposes like speeding up proof generation.

Example:

2) Mistake in the Fiat-Shamir Transformation

There are instances where vulnerabilities arise when converting to non-interactive proofs. Moreover, it's not uncommon for the security degradation in the Fiat-Shamir process to go unchecked. Discussions frequently revolve around a conjectured security, which assumes the post-transformation bit security is the same as the pre-transformation bit security.

Example: https://blog.trailofbits.com/2022/04/15/the-frozen-heart-vulnerability-in-bulletproofs/

3) Trusted Setup

For ZKP schemes that require a trusted setup, if the toxic waste is misused, fake proofs might get verified, or secrets could be leaked. Especially concerning fake proofs, due to the entire scheme's zero-knowledge nature, external observers cannot detect the fakes. Therefore, for protocols employing zero-knowledge proofs for privacy, detecting attacks and estimating their scale is challenging, potentially leading to catastrophic long-term damage. Using ZKPs without a trusted setup or

conducting a universal setup is decisively crucial for such applications.

4) Mistakes in Circuit Design

Even if the ZKP is secure, those designing circuits using it can introduce vulnerabilities at that stage, which is quite common. A typical example in Circom would be confusing substitutions and constraints. Mistakes in circuit design, much like bugs in smart contracts, are expected to be frequent. Consequently, in the cryptocurrency industry, there's a thriving practice of code audits specifically for ZKPs.

12 Applications of ZKPs

About ZCash:

ZCash, launched in 2016, is a cryptocurrency focusing on privacy, aiming to completely conceal the sender, receiver, and transfer amount. It was one of the first attempts in cryptocurrency to employ zero-knowledge proofs, using Groth16.

About pay2sudoku:

This was an application experiment on Bitcoin using zero-knowledge proofs by core developer Gregory Maxwell. It allowed someone who knew the solution to a Sudoku puzzle to prove it using a zero-knowledge proof and receive coins. Notably, since the Bitcoin script doesn't have a function to verify zero-knowledge proofs, the Sudoku challenge was guaranteed using a zero-knowledge proof by substituting knowing the solution to the puzzle with knowing the preimage of a hash. Therefore, unlike conventional zero-knowledge proofs, where the circuit creator doesn't necessarily need to know the answer that meets the constraints, the pay2sudoku scheme presupposes that the challenger knows the solution to the Sudoku.

About Tornado Cash:

Tornado Cash is a mixing application on Ethereum, created with a focus on privacy like ZCash. It was written and deployed with Circom and was the first zero-knowledge proof application to attract many users.

How it works: https://github.com/tornadocash/tornado-core

About Aztec Protocol/zkMoney:

zkMoney is a private cryptocurrency on Ethereum developed by the Aztec team. The main distinction from ZCash is whether it's Layer 1 or an application on Layer 1. Ideally,

expressing it in ERC20 would be optimal for user-friendliness, but this is technically challenging, so it was implemented using an entirely new token standard.

About Mina Protocol:

Mina is a Layer 1 focused on zero-knowledge proofs and has an extremely small blockchain size, even smaller than Bitcoin's. It uses Cyclic Recursive ZKP, making the entire blockchain verification instantaneous. Similar methods could also be applied to Bitcoin's full nodes, and several such projects are currently identified.

About ID & KYC:

IDs using ZKP and blockchain are often referred to as DID/KYC. They range from those complying with regulations like KYC to non-compliant ones. Project Zuzalu, which began under the concept of a Network State aiming for a position equivalent to a nation by appointing diplomats to a DAO, used ZKP for its virtual nation member passports.

About Selective Disclosure:

With ZKP, selective disclosure can be implemented by introducing a trapdoor in the proof. Digital credentia

About Credit Score:

About NFT:

Since NFTs serve as proofs of ownership, their owners' addresses and on-chain histories are easily trackable by many. ZKP is applied to anonymize possession and address this issue.

About Data Availability Sampling:

Conceived by Vitalik Buterin for the purpose of completing zkRollup and detailed by Bankrad Fiest, Data Availability Sampling is a mechanism to prove data retention for a certain period. It allows nodes to efficiently prove that they're holding data fragments, utilizing two-variable polynomial commitments.

About Bridges:

With ZKP, it's possible to construct trustless 2-way peg bridges between many public blockchains. However, both chains must be capable of verifying ZKP. If generic smart contracts are implemented, this condition can be automatically met. Conversely, a trustless bridge between Ethereum and Bitcoin using ZKP hasn't been discovered yet

and is said to require Witness Encryption.

About zkML:

zkML refers to systems that guarantee, through NIZK, that a model was constructed using a specific algorithm and dataset.

About AML:

The "Proof of No Crime" concept, proposed and implemented by Tornado Cash, is a mechanism that, while possessing the properties of mixing, allows one to prove that no funds were received from a specific address. It's a system that seeks a balance between privacy and AML.

Secret Voting:

Games:

zkOracle

funny category:

12.1 zkRollup

12.1.1 General

・Definition of zkRollup

zkRollup is a network that secures the bridge between Layer1 and off-chain using ZKP while retaining the security of Layer1.

・History of zkRollup

It was conceived by Vitalik Buterin in 2018, and currently, there are more than 10 networks that are its improved versions. Before Vitalik Buterin's idea, there was a proposal called Plasma snap which applied zkp to the Layer2 solution Plasma, and Barry Whitehat performed the MVP implementation. In 2019, Loopring first implemented and launched a DEX using zkRollup, and in 2020, zkSync launched the remittance network zkSync v1. In 2023, the Polygon team developed zkEVM and launched a zkRollup fully compatible with Ethereum VM, i.e., Solidity.

・Classification of zkRollup

zkRollup can be classified into the following three based on the uniqueness of the state:

Stateful zkRollup - Manages bit commitments of the state of users and smart contracts with Layer1 smart contracts. Examples: Starknet, zkSync Era, Polygon zkEVM, Scroll, Taiko, Loopring

Succinct zkRollup - Allows everyone to calculate bit commitments of the state of users and smart contracts from Layer1 data. Example: Sovereign Rollup

Stateless zkRollup - Users only compute their own bit commitment without fetching from Layer1 data. Example: Intmax

Furthermore, the most prominent zkRollup, the Stateful zkRollup, can be classified as:

Type1 Exactly equivalent to the Ethereum Network. Example: Taiko

Type2 Fully compatible with EVM bytecode but not equivalent to Ethereum Network. Example: Scroll

Type3 Mostly compatible with EVM and many Solidity codes are portable. Example: Polygon zkEVM

Type4 Compatibility with EVM is not guaranteed, and there's no guarantee that many Solidity codes are portable. Examples: StarkNet, zkSync

These classifications by zkEVM are important in the Ethereum community due to their impact on security during porting from Solidity code.

Additionally, there are zkp solutions that do not meet the requirements of zkRollup:

・zkPorter

A solution proposed by zkSync to sacrifice zkRollup security to achieve scalability. It was proposed to offset the DA cost bottleneck in zkRollup by a new token-carrying network, eliminating the need to pay Ethereum Layer1 DA costs. A DA attack by a coalition of 2/3 token holders can freeze user assets but cannot confiscate them, so there is no incentive for the attacker. Since the security assumption of a new token is included, it does not inherit the security of Ethereum Layer1.

While not classified as zkRollup, it's debatable whether it's a Layer2 or a sidechain:

・Validium

A system developed by zkSync. It is a mechanism that trusts a validator group not to carry out a DA attack, foregoing DA cost payments.

・Volition

Proposed by Starkware, it is a mechanism where individuals can choose the trustlessness level, i.e., security level, of DA storage. There's a trade-off between price and security.

-zkRollup Concerns and Solutions

11.1-2 Rollup != Bridge Argument

If a token is natively issued on Rollup (Layer2) and is not the subject of a bridge with Ethereum Layer1, there has been a claim that this token may not inherit Ethereum Layer1's security if it conducts its own forks or governance.

However, even tokens on Ethereum Layer1 that have their governance or references with projects, or lack liquidity, often do not represent Layer1's trustlessness. It was suggested that Rollup inherits not only the security of Ethereum Layer1 but also the reduced security properties by each community, affirming Rollup's role as a pure extension of Ethereum. Currently, this argument isn't seen as very valuable, so it rarely comes up.

11.1-3 Does zkRollup use Zero-Knowledge?

Most zkRollups are not confidential. Therefore, some argue that zkRollups only utilize the non-interactive nature of zkSNARKs. The validity of this claim boils down to:

"When there are many inputs that, while not a security issue if made public, are kept confidential for efficiency, is ZK being utilized?"

In other words, in Stateful zkRollups, the prover hides data not because they want to but because making it public wastes computational resources.

11.1-4 Decentralization of zkRollup's block producers

Many Stateful zkRollups have not decentralized block producers due to the high computational cost of zkp and the risk of bugs. While this doesn't pose a security risk given zkRollup's inheriting properties, it does raise concerns about service continuity.

11-1-5 Potential for Decreased Security via proto-danksharding/danksharding

The use of time-limited data, such as those provided by proto-danksharding, for periods like one month, is a very novel approach in the blockchain industry. With this, while effectively preventing block withholding attacks, rollup nodes can transfer Data Availability (DA) from Layer1. This is possible within rollups because if even one node is an honest node, it can prevent a DA attack. This is known as the 1/N security assumption, and it's considered a method to scale Layer2 trustlessly. Whether writing all data into Layer1's call data or moving the data storage to rollup nodes through proto-danksharding, both approaches operate on the 1/N assumption. However, the value of N differs significantly between them. When writing to Layer1, N might be the number of Ethereum nodes, say 5000, but when using proto-danksharding, N might be the number of candidate nodes, perhaps around 20.

For major Layer2/Rollups with high public trust and significant value concentration, maintaining a large N might be possible. However, for rollups crafted through mechanisms like "Rollup as a service," it's challenging to predict what this N number might be.

13. Security considerations

13. Choosing a Stack

- Small proofs & fast verify → pairing/KZG (Groth16/Plonk).
- Transparent & long traces → FRI/STARK.
- Flexible recursion/folding → Halo2, Plonky2/3, Nova/HyperNova.

14. Ecosystem Snapshot (2024–2025)

- Plonky3 >2M Poseidon2 hashes/s on M3 Max (reported); powering SP1/Valida.
- Nova/HyperNova/MicroNova advance folding/IVC; on-chain verification improvements.
- zkVMs maturing; universal verifiers and aggregation.

15. Mathematical Derivations (Concise)

15.1 Groth16 via QAP

Let $A_w(X)=\sum w_i A_i(X)$, $B_w(X)=\sum w_i B_i(X)$, $C_w(X)=\sum w_i C_i(X)$, $t(X)=\prod(X-r_j)$. Require $A_w(\tau)\cdot B_w(\tau) - C_w(\tau) = h(\tau)\cdot t(\tau)$. Proof = ($A\in G1$, $B\in G2$, $C\in G1$); verify via three pairings.

15.2 KZG Evaluation Proof

Commit $c=g^{\varphi(\tau)}$; open at a with $y=\varphi(a)$. $q(X)=(\varphi(X)-y)/(X-a)$; $\pi=g^{q(\tau)}$; verify $e(c/g^y, g) = e(\pi, g^{\tau}/g^{a})$. Constant size.

15.3 Sumcheck (Sketch)

Prove $S=\sum_{x\in\{0,1\}^n} g(x)$ by recursively binding challenges and reducing arity; finalize with direct checks.

15.4 FRI (Low-Degree Test)

Commit to evaluations; apply random linear combinations and downsample; queries check proximity to low-degree polynomials.

16. Measurements & Representative Sizes

Table 16.1 — Proof sizes (typical configurations)

System	Curve / Security	Proof contents	Compressed size	Uncompressed size (if on-chain)
Groth16	BN254 (~100–128-bit)	$A\in G1$, $B\in G2$, $C\in G1$	~128 B (32+64+32)	≈256 B (64+128+64)
Groth16	BLS12-381 (128-bit+)	$A\in G1$, $B\in G2$, $C\in G1$	~192 B (48+96+48)	≈384 B (96+192+96)
KZG eval proof	BLS12-381	$\pi\in G1$	48 B	96 B (uncompressed)
IPA PC (poly eval)	Curve-group	O(log n) elements	logarithmic	logarithmic
STARK/FRI	Hash-based	Queries + commitments	100–250 KB (typical)	same

Table 16.2 — EIP-4844 blob economics (constants)

Parameter	Symbol	Value
Bytes per field element	BYTES_PER_FIELD_ELEMENT	32

Field elements per blob	FIELD_ELEMENTS_PER_BLOB	4096
Bytes per blob	—	131,072 bytes
Blob gas per blob	GAS_PER_BLOB	2^17 = 131,072
Target / Max blobs per block	—	3 target / 6 max

Example: cost per blob = base_fee_per_blob_gas × GAS_PER_BLOB. At 1 gwei per blob gas ⇒ 131,072 gwei per blob (≈1.31e-4 ETH).

17. On-Chain Verification Notes

- Groth16 on BN254: mature Solidity verifiers (pairing precompiles).
- KZG on BLS12-381: point-evaluation precompile (EIP-4844).
- IPA verifiers: cost dominated by MSMs; batch openings recommended.

18. References (Selected)

[R1] Groth, 2016 — On the Size of Pairing-based NIZKs (ePrint 2016/260).

[R2] KZG — Constant-size polynomial commitments overview.

[R3] EIP-4844 — Spec parameters & blob fee market.

[R4] Ethereum.org — Dencun FAQ (activation 13-Mar-2024; blob lifetime).

[R5] Blocknative — Blob congestion event; still cheaper than calldata in practice.

[R6] Nova 2021/370; SuperNova 2022/1758; HyperNova 2023/573; MicroNova 2024/2099 (ePrint).

[R7] Halo2 Book — lookup arguments; PlonKup ePrint 2022/086.

[R8] Starknet S-two benchmarks (proof sizes).

[R9] Plonky3 — Polygon Labs blog (2024-12-12) & CoinDesk coverage (2024-07-16).

[R10] Celestia Docs — DAS/NMT; EIP-7594 PeerDAS (Last Call 2025).[R11] zkVerify doc — EC point sizes (BN254/BLS12-381, uncompressed).

[R12] Bulletproofs — ePrint 2017/1066 (logarithmic IPA).

Appendix A — Glossary

ZKP, NIZK, CRS/SRS, PCS, R1CS, Plonkish, AIR, CCS, FRI, IPA, IVC, folding, recursion, zkVM, DA, lookup, MSM, FFT, ROM, AGM.

Appendix B — Comparison Tables

Family	Setup	Assumptions	Proof size	Verify time	Prover cost	Notes
Pairing/KZG	SRS (circuit/universal)	AGM/ROM, DLOG	Smallest	Few pairings	FFT+MSM heavy	Mature on-chain
IPA	Transparent/universal	DLOG	O(log n)	MSMs	Moderate–High	Good recursion
FRI/STARK	Transparent	Hash/LDT	100–250 KB+	Queries	Scales well	VM traces
Folding/IVC	Depends on PCS	Sumcheck + PCS	Aggregated	Succinct	Low per-step	Streaming/PCD
Front-End	Primary use	Lookups	Trace style	Strengths	Considerations	
R1CS	Early SNARKs	Limited	Gate constraints	Simple	Bit ops costly	
Plonkish	SNARKs w/ custom gates	Native	Polynomial constraints	Efficient hashing/lookups	Design complexity	
AIR	STARK systems	Common	Time-step transitions	VM traces	Larger proofs	
CCS	Unified	Via gadgets	Flexible	IVC-friendly	Advanced tooling	

Appendix C — Worked Examples (Sketches)

Fiat–Shamir Transcript: challenge = H(commitments || instance || domain). Lookup arguments: sort-merge table + permutation products.